

Day 86: MySQL Basics and Connecting Python to MySQL

1. Day Number

Day 86

2. Topic Name

MySQL and Python Database Connection

Today you will learn how a Python program can communicate with a **MySQL database**.

You already know basic SQL such as:

```
SELECT
INSERT
UPDATE
DELETE
```

Now you will see how Python can send those SQL commands to MySQL.

3. Connection

Yesterday you learned how SQL can:

- read rows with `SELECT`
- add rows with `INSERT`
- change rows with `UPDATE`
- remove rows with `DELETE`
- combine tables with `JOIN`

Until now, you were mainly thinking about SQL by itself.

Today the idea becomes:

```
Python program
  ↓
sends SQL query
  ↓
MySQL database
  ↓
returns result
  ↓
Python program
```

This is how many real applications work.

4. Important Topics

MySQL

MySQL is a relational database management system.

It can store tables such as:

```
customers
products
orders
employees
```

For example:

```
products
-----
id | name      | price
1  | Keyboard  | 1200
2  | Mouse     | 500
3  | Monitor   | 9000
```

Python can connect to MySQL and ask it to return these rows.

Database Server

MySQL usually runs as a **database server**.

Think of the server as a program responsible for:

- storing the database
- receiving requests
- executing SQL
- returning results

Your Python application does not normally read MySQL's internal files directly.

Instead, it communicates with the MySQL server.

Connection

Before Python can send SQL, it needs a **connection**.

Conceptually:

```
connection = connect_to_mysql(...)
```

The connection tells Python:

```
| "I want to communicate with this MySQL server."
```

A connection normally needs information such as:

```
host
username
password
database name
```

For local learning, the host is often:

```
localhost
```

which means:

| "The MySQL server is running on my own computer."

Cursor

A **cursor** is an object Python uses to send SQL commands through the database connection.

Conceptually:

```
cursor = connection.cursor()
```

Then:

```
cursor.execute("SELECT ...")
```

Think of it like this:

```
Connection → opens communication
Cursor → sends SQL through that communication
```

You do not need to understand the internal details yet.

Query

A **query** is the SQL instruction you send.

For example:

```
SELECT name, price
FROM products;
```

Python can store this SQL as a string:

```
query = "SELECT name, price FROM products"
```

and then ask the cursor to execute it.

Result

After MySQL runs a `SELECT` query, it can return rows.

For example:

```
('Keyboard', 1200)
('Mouse', 500)
('Monitor', 9000)
```

Python can retrieve those rows and process them.

5. Foundational Notes

There are two different languages involved today.

Python

Python controls your application logic.

For example:

```
print()
for
if
try
except
```

SQL

SQL communicates with the database.

For example:

```
SELECT name
FROM products;
```

Python does not replace SQL.

Instead, Python can **send SQL to MySQL**.

A useful mental model is:

```
Python = application logic

SQL = database instructions

MySQL = stores and manages the data
```

Beginner-Level Setup

To practice locally, you generally need two things:

1. A running MySQL server
2. A Python MySQL connector/library

After installing MySQL, you might create a small database such as:

```
shop_db
```

containing a table like:

```
products
```

Python also needs a package that knows how to communicate with MySQL.

One common beginner option is:

```
mysql-connector-python
```

It can usually be installed with:

```
pip install mysql-connector-python
```

Then Python can import it conceptually with:

```
import mysql.connector
```

You do not need connection pooling, advanced server configuration, or deployment settings at this stage.

6. Client and Database-Server Communication

Suppose you have a Python program.

That Python program is the **client**.

MySQL is the **database server**.

The communication looks like:

```
Python client
  |
  | "SELECT name FROM products"
  ↓
MySQL server
  |
```

```
| finds matching rows
↓
Python receives rows
```

Imagine going to a library desk.

You ask:

```
| "Please give me the books written by this author."
```

The librarian searches the library and gives you the matching books.

Similarly:

```
Python → sends request

MySQL → searches stored data

MySQL → sends result back
```

Python can then display or process that result.

7. Easy Example

Suppose MySQL contains this table:

```
products
-----
id | name      | price
1  | Keyboard  | 1200
2  | Mouse     | 500
3  | Monitor   | 9000
```

You want Python to run:

```
SELECT name, price
FROM products;
```

The Python-side process conceptually looks like:

```
connect to MySQL

create cursor

send SELECT query

get returned rows

for each row:
```

```
print row

close connection
```

A simplified code pattern might look like:

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="your_username",
    password="your_password",
    database="shop_db"
)

cursor = connection.cursor()

query = "SELECT name, price FROM products"

cursor.execute(query)

rows = cursor.fetchall()

for row in rows:
    print(row)
```

For now, focus on the **flow** rather than memorizing every line.

The important sequence is:

```
connect
→ cursor
→ execute query
→ fetch result
→ process rows
```

8. Problem Statement

Imagine you have a MySQL database called:

```
shop_db
```

Inside it is a table:

```
customers
```

with data such as:

customer_id	name	city
1	Asha	Bengaluru
2	Ravi	Chennai
3	Meera	Delhi

Your task is to conceptually write a small Python program that:

1. connects to the MySQL database
2. creates a cursor
3. runs this type of SQL query:

```
SELECT name, city
FROM customers;
```

4. retrieves the returned rows
5. prints each customer
6. handles a basic connection failure

Keep the program very small.

9. Concepts Used

You will use:

- Python
- MySQL
- relational database
- database server
- database connection
- client/server idea
- cursor
- SQL query
- `SELECT`
- returned rows
- `execute()`
- `fetchall()` concept
- loop
- basic exception handling

10. Thought Process

Before coding, think through the communication.

Step 1: Where is the data?

The customer data is stored inside:

```
MySQL
```

Therefore Python needs to communicate with MySQL.

Step 2: What must happen first?

Python cannot immediately execute a query.

First it needs:

```
connection
```

Step 3: What sends the SQL?

Create a:

```
cursor
```

Step 4: What information do we want?

We want:

```
customer name  
customer city
```

So the SQL will use:

```
SELECT name, city
```

Step 5: What happens after execution?

MySQL returns matching rows.

Python must retrieve them.

Conceptually:

```
rows = cursor.fetchall()
```

Step 6: What should Python do with them?

Loop through the returned rows:

```
for each returned row  
    print the row
```

11. Beginner-Friendly Pseudocode

```
START

import MySQL connector

TRY:
    connect to MySQL server

    create a cursor

    create SELECT query

    execute query

    fetch all returned rows

    FOR each row:
        print row

    close cursor
    close connection

EXCEPT connection error:
    print a simple error message

END
```

The main pattern to remember is:

```
Connect
↓
Cursor
↓
Execute
↓
Fetch
↓
Process
↓
Close
```

12. Suggested Solving Approach: Python Database Client

Use a simple **Python database client approach**.

Step 1

Import the database connector.

```
import mysql.connector
```

Step 2

Create the database connection.

Think about the required values:

```
host  
user  
password  
database
```

Step 3

Create a cursor from the connection.

```
connection  
  ↓  
cursor
```

Step 4

Store a simple SQL query in a Python string.

For example:

```
SELECT name, city  
FROM customers;
```

Step 5

Ask the cursor to execute the SQL.

Step 6

Fetch the results.

For a small beginner exercise, you can think about:

```
fetchall()
```

as:

```
| "Give me all rows returned by my SELECT query."
```

Step 7

Loop through the rows and print them.

Step 8

Close the database resources when finished.

13. Easy Edge Cases

Connection Failure

A connection might fail because:

- MySQL is not running
- the username is wrong
- the password is wrong
- the database name is wrong
- the server cannot be reached

Instead of letting the program suddenly crash, you can handle the error with a simple `try` and `except`.

Conceptually:

```
try:
    connect to database

except:
    print("Could not connect to database")
```

Later you can learn more precise database exceptions.

Empty Result

Suppose you execute:

```
SELECT name, city
FROM customers
WHERE city = 'Jaipur';
```

but there are no customers from Jaipur.

The database has not necessarily failed.

It may simply return:

```
no rows
```

Your Python program should understand that:

```
empty result ≠ connection error
```

Conceptually:

```
if there are rows:
    print them
else:
    print("No customers found")
```

14. Expected Flow

Assume your MySQL table contains:

customer_id	name	city
1	Asha	Bengaluru
2	Ravi	Chennai
3	Meera	Delhi

Your Python program sends:

```
SELECT name, city
FROM customers;
```

The flow is:

```
Python starts
  ↓
Connects to MySQL
  ↓
Creates cursor
  ↓
Sends SELECT query
  ↓
MySQL executes query
  ↓
MySQL returns rows
  ↓
Python fetches rows
  ↓
Python loops through rows
  ↓
Rows are printed
```

A possible output could conceptually look like:

```
('Asha', 'Bengaluru')
('Ravi', 'Chennai')
('Meera', 'Delhi')
```

Later, you can format these tuples into more user-friendly output.

15. Hint Only

Start by remembering this sequence:

```
connection
→ cursor
→ execute
→ fetch
→ loop
→ close
```

Your SQL query can stay extremely simple:

```
SELECT name, city
FROM customers;
```

For the Python structure, think:

```
import connector

try:
    create connection
    create cursor
    execute SELECT
    fetch rows

    for each row:
        print row

except:
    show connection error
```

Do not worry about advanced configuration yet.

The key idea for **Day 86** is:

Python acts as the client, MySQL acts as the database server, and a database connector allows Python to send SQL queries and receive results.